System Design Interview vol. 2 (PDF)

05
Design A Rate Limiter

In this system, we will design a rate limiter. Which algorithm is the best for its implementation is a popular question. We will discuss various algorithms and their pros and cons. We will also talk about how to scale the rate limiter to handle large volumes of traffic. We will also talk about how rate limiters are deployed.

Step 1 - Understand the problem and establish design scope

- Client-side rate limiting
- Rate limiting based on IP address, user ID, or other properties
- Rate limiting in a distributed environment
- Rate limiter as a service

In this system, we are interested in a server-side API rate limiter.

Requirements

- Accurately limit excessive requests.
- Low latency. The rate limiter should not slow down HTTP response time.
- Efficient memory usage.
- Distributed rate limiting. The rate limiter can be shared across multiple servers or processes.
- Exception handling. Show clear exceptions to users when their requests are throttled.
- High fault tolerance. If there are any problems with the rate limiter, the rate limiter does not affect the entire system.

Step 2 - Propose high-level design and get buy-in

Where to put the rate limiter?

- Client-side implementation. Client-side rate limiting is not a good choice because we can easily forge client requests.
- Server-side implementation. Generally, a server-side rate limiter is used. We will discuss rate limiting algorithms that impose no control over the hacking attacks, however, the server-side rate limiter is more secure than client-side rate limiting.

Figure 1

Besides the client and server-side implementations, there is an alternative way. Instead of putting a rate limiter on the API server, we create a rate limiter middleware and put it on the API server. Figure 2 shows the new design.

Figure 2

Let us use a cloud-native 7 layer Internet flow as the rate limiting point in this design. Assume our API server is frequently accessed by users, which is a common scenario for most tech companies. We choose to put the rate limiter on the server side because the server side implementation is under our control. We also put a rate limiter in front of the API server to ensure that the rate limiting is done before the traffic reaches the API server.

Figure 3

Cloud microservices do have become widely popular and rate limiting is usually implemented within a component called API gateway. API gateway is a cloud-native service that supports rate limiting, SSL termination, authentication, IP whitelisting, servicing static content, service discovery, monitoring, and other functionalities. We do not need to install our own rate limiter if we choose to use API gateway.

Rate limiting algorithms are simple, and it is relatively practical to implement them on the server side. We choose to use the server-side implementation and API gateway for our design. We will discuss the details of rate limiting algorithms in the next section.

- Generally, API gateway is a middleware that supports rate limiting, SSL termination, authentication, IP whitelisting, servicing static content, service discovery, and other functionalities. Cloud vendors usually provide API gateway as a cloud service. Some popular API gateway offerings are Amazon API Gateway, Google Cloud Gateway, Kong, Apigee, Tyk, and Zuul.
- Amazon Web Services provides managed services for many common functionalities such as authentication, monitoring, and rate limiting. Instead of using API gateway, we can also use AWS API Gateway to manage our API. Amazon API Gateway is a fully managed service that supports rate limiting, SSL termination, authentication, IP whitelisting, and monitoring. However, it does not support servicing static content, service discovery, and other functionalities. Amazon API Gateway is a good choice if we are looking for a cloud-native solution that supports most of the functionalities we need in our design.
- Rate limiting is a very important feature to protect our APIs from being overwhelmed by requests. We can implement rate limiting in our own server or we can use a third-party API gateway service. Amazon API Gateway is a managed service that supports rate limiting. Some popular open source API gateway offerings that support rate limiting are Kong, Zuul, and Envoy. We can also use API gateway as a middleware to implement our own rate limiter.

Step 3 - Design deep dive

Algorithms for rate limiting

Rate limiting can be implemented using different algorithms, and each of them has its own pros and cons. Even though this chapter does not focus on algorithms, understanding how it works at high level helps to choose the right algorithm or combination of algorithms to fit our use case. Here are a few algorithms:

- Token bucket
- Leaking bucket
- Fixed window counter
- Sliding window log
- Sliding window counter

Token bucket algorithm

The token bucket algorithm is widely used for rate limiting. It is simple, well understood and commonly used by internet companies. Both Amazon and Stripe use this algorithm to shape their API requests.

The token bucket algorithm works as follows:

- A token bucket is a container that has pre-defined capacity. Tokens are put in the bucket every a certain period of time. Some people also refer to the token bucket as a token pool. For simplicity, we use the token bucket in this book.
- Each request consumes one token. When a request arrives, we check if there is enough tokens in the bucket. If there is at least one token left in the bucket, we take one token from the bucket and the request is forwarded to the server. If there is not enough tokens in the bucket, the request is dropped.

Figure 4

Figure 4 illustrates how token consumption, refill, and rate limiting logic work in this example. Our token bucket can at most fit 4 tokens, and 2 tokens are added to the bucket every second. Three tokens are left in the bucket after 8 requests are processed every second. The next two requests can be processed because there are enough tokens in the bucket. However, we will see that the rate limiter drops the request if there is not enough tokens in the bucket.

The token bucket algorithm takes two parameters:

- The bucket size: it is the maximum number of tokens allowed in the bucket.
- The refill rate: it defines how many tokens are put into the bucket every second.

How many buckets do we need? It depends on the rate limiting rules. Here are few examples.

- It is possible to have different buckets for different API endpoints. For example, if the API allows a maximum of 1 request per second, a user can send 150 requests per second to the post API, and 10 requests per second to the home API. The rate limiter can create 2 buckets for each user, one for post API and the other one for home API. The rate limiter can also put all requests to the same API endpoint into the same bucket.
- If the system is a global rate limiter, we can create a bucket for each IP address.
- If the system is a global rate limiter, we can also create a bucket for the system.

Leaking bucket algorithm

The leaking bucket algorithm is similar to the token bucket except that requests are processed at a fixed rate. It is usually implemented with a FIFO queue. The algorithm works as follows:

- When a request arrives, the system checks if the queue is full. If it is not full, the request is added to the queue.
- Otherwise, the request is dropped.
- Requests are taken from the queue and processed at a fixed rate.

Figure 5

Figure 5 illustrates the leaking bucket algorithm. Assume the queue size is 4, and the system can process 2 requests per second. If 8 requests arrive at the rate limiter, the rate limiter puts 4 requests into the queue and drops the other 4 requests. The 2 requests at the front of the queue are pulled out of the queue and processed. The other 2 requests are still waiting in the queue to be processed. The rate limiter will pull 2 more requests from the queue and process them 1 second later. If a new request arrives, it will be added to the queue as long as there is enough space in the queue.

Leaking bucket algorithm takes the following two parameters:

- The bucket size: it is equal to the queue size. The queue holds the requests to be processed at a fixed rate.
- Outflow rate: it defines how many requests can be processed in a fixed time period.

Pseudocode of a minute counter, per rate limiter for the entire IP address:

```
Initialize a hash table, key: IP address, value: counter
Initialize a timer that expires every 1 minute
Upon receiving a request from IP address:
    If counter > rate limit:
        Drop the request
    Else:
        Forward the request to the server
    Increment counter by 1
Upon timer expiration:
    Reset all counters
```

Fixed window counter algorithm

Fixed window counter algorithm works as follows:

- The algorithm divides the timeline into fixed-size time windows and assign a counter for each window.
- Each request increments the counter by one.
- Once the counter reaches the predefined threshold, new requests are dropped until a new time window starts.

Let's see how this algorithm works in a specific example. Assume the time unit is 1 second and the system can accept a maximum of 3 requests per second. In each second, 3 requests are allowed. If 3 requests are received during a second, all requests following these 3 requests are dropped in that second.

Figure 6

Leaking bucket algorithm takes the following two parameters:

- Window size: it is the time window unit in which the maximum number of requests are processed at a fixed rate.
- Window size: it defines how many requests are processed in a window.

Pseudocode of a fixed window counter, per rate limiter for the entire IP address:

```
Initialize a hash table, key: IP address, value: counter
Initialize a timer that expires every 1 second
Upon receiving a request from IP address:
    If counter > rate limit:
        Drop the request
    Else:
        Forward the request to the server
    Increment counter by 1
Upon timer expiration:
    Reset all counters
```

The advantage of fixed window counter algorithm is it is easy to understand. Unfortunately, it has a critical flaw. Imagine if we allow 10 requests per second, and all 10 requests in the first half of a second and all 10 requests in the second half of a second are processed by the server. The server will be busy for 1 second, but the 10 requests we allowed for that second are processed in 0.5 second. The other 10 requests we allowed for the second half of a second will be processed in the next second. In other words, we are processing 20 requests in 0.5 second. This is not what we want to do. We want to spread out the processing of the requests evenly over time.

Sliding window log algorithm

Sliding window log algorithm fixes the issue of the fixed window counter algorithm. It works as follows:

- The algorithm keeps a time stamp for each request. Time stamps are usually kept in a sorted set or a hash table.
- When a new request comes in, we add its time stamp to the sorted set.
- In the sorted set, we can remove any time stamps of previous windows. These time stamps are no longer in the window we are looking at.
- After adding the new request, we compute the sum of the number of time stamps in the window. If the number of time stamps is less than or equal to the allowed count, the request is accepted. Otherwise, it is rejected.

Let's see how this algorithm works in a specific example. Assume our rate limiter allows 2 requests per minute. Normally, we can use Unix time to represent time. For simplicity, we use a string representing the minute and second. For example, 1:01 represents the first minute of the first hour. Figure 7 shows the sorted set at various time stamps.

Figure 8

In this example, the rate limiter allows 2 requests per minute. Usually, every incoming request is stored in the sorted set. However, there are 2 exceptions. The second request is rejected because it is the same time stamp as the first request, and we can only have 2 time stamps in the sorted set. The third request is also rejected because the time stamp is 1:01, which is the same time stamp as the first request, and the time stamp of the first request is still in the sorted set. The fourth request is accepted because the time stamp is 1:02, and the time stamp of the first request is now outside the window. We can remove the first time stamp from the sorted set. The fifth request is also rejected because the time stamp is 1:02, and there are already 2 time stamps in the sorted set.

Pseudocode of a sliding window log rate limiter:

```
Initialize a hash table, key: IP address, value: sorted set
Upon receiving a request from IP address:
    Add time stamp to the sorted set
    Remove all time stamps in the sorted set that are older than one minute
    If size of the sorted set > rate limit:
        Drop the request
    Else:
        Forward the request to the server
```

Advantage of sliding window log algorithm is it is very accurate. The number of time stamps in the window never exceeds the rate limit. However, it consumes a lot of memory because it keeps a sorted set in memory.

Sliding window counter algorithm

The sliding window counter algorithm is a hybrid approach that combines the fixed window counter and sliding window log. The algorithm can be implemented in two different ways. In this section, we will show the high-level idea of the algorithm. We will show the detailed implementation in the later section "Detailed design".

Suppose the rate limiter allows 7 requests per minute, and 5 requests were received in the previous minute and 3 requests were received in the current minute. For a new request, the sliding window counter calculates the number of requests based on a weighted average of the previous window and the current window. The formula is as follows:

Figure 9

Assume the rate limiter allows 7 requests per minute, and there were 5 requests in the previous minute and 3 in the current minute. For a request that arrives at a 30% position in the current minute, the number of requests in the rolling window is calculated by 5 * 0.70 + 3, where 5 is the number of requests in the previous minute, 0.70 is the overlap between the rolling window and previous minute, and 3 is the number of requests in the current minute. In this example, the number is 6.5. Depending on the use case, we can also round or floor the number. We choose to round the number for our use case.

Since the rate limiter allows a maximum of 7 requests per minute, the current request can be accepted. Because the rate limiter allows a maximum of 7 requests per minute, an additional request can be accepted in the current window if it is calculated to be no greater than 7 in the future. Assuming the request takes another 1 minute to complete, and the distribution of requests in the previous minute is the same as the current minute, the number of requests in the rolling window will be 5 * 0.70 + 3 = 6.5 in the next minute. Based on this calculation, we can still accept 3 more requests in the current window.

Pseudocode of a sliding window counter rate limiter:

```
Initialize a hash table, key: IP address, value: counter
Initialize a timer that expires every 1 minute
Upon receiving a request from IP address:
    If counter > rate limit:
        Drop the request
    Else:
        Forward the request to the server
    Increment counter by 1
Upon timer expiration:
    Reset all counters
```

Sliding window counter algorithm is a very practical approach because it gives the benefit of being able to scale quickly with fast traffic patterns. Assume our system does not allow any spikes of traffic. Based on our previous calculation, we can add more servers to our system when the number of requests in a window is expected to be 0.7 * 7 = 4.9 requests. Sliding window counter algorithm is also used in Cloudflare's rate limiter. Cloudflare's rate limiter has processed 400 million requests from over 2 million websites. Cloudflare confirms that only 0.003% of requests are wrongly allowed or rate limited among the 400 million requests they have processed.

High level architecture

Rate limiting algorithms are relatively simple, and the details of each algorithm are well known. The question we need to ask is where should we put the rate limiter? Is it possible to put it in our backend server? Is it a good idea to put our own rate limiter in front of our API servers? Is it a good idea to use our own API gateway? The answer to these questions depends on how far we have gone in our software development journey. If our company is small and our traffic is not high, we can put the rate limiter in our backend server. If our company is growing and we want to use a third-party API gateway service, we need to make sure the API gateway service supports our rate limiting algorithm. If we want to have our own rate limiter, we need to make sure it is easy to implement a rate limiter in our backend server. In the end, we need to have a rate limiter that is fast and efficient. We want our rate limiter to be free from any language or framework limitations, because we may want to add more clients such as mobile apps in the future. We want our rate limiter to be flexible to cope with any rate limiting algorithm we want to use in the future. In summary, we want to have our own rate limiter in our backend server.

Rate limiting can be implemented in the application layer and in the network layer. We usually choose to implement rate limiting in the application layer. In the application layer, it is easy to implement a rate limiter in the server side if we have the server-side language and skills. However, it may be more efficient to implement rate limiting in the network layer. For example, we can block IP addresses that are in the rate limiting state in the network layer. In our design, we choose to implement our rate limiter in the application layer because it is easy to implement and it is compatible with the cloud environment.

Rate limiting can be implemented in the application layer. Middleware is a popular way to implement rate limiting in the application layer. Middleware is a piece of code that intercepts incoming requests. We can add a middleware to our API server to control the rate limiting. Middleware is modular and pluggable. We can add a middleware to our API server without changing any code. We can also add a third-party middleware for rate limiting. We can also add our own middleware if we have the time and resources to do so.

Rate limiting can be implemented in the application layer. Middleware is a popular way to implement rate limiting in the application layer. Middleware is a piece of code that intercepts incoming requests. We can add a middleware to our API server to control the rate limiting. Middleware is modular and pluggable. We can add a middleware to our API server without changing any code. We can also add a third-party middleware for rate limiting. We can also add our own middleware if we have the time and resources to do so.

Let's use middleware to add our rate limiter. We choose middleware because it is easy to implement, it is efficient, it is flexible, and it is not limited by any language or framework. We can add our own middleware if we want to add more features in the future. We can also add a third-party middleware if we want to use a third-party API gateway service. We can also add our own middleware if we want to add more features in the future. We can also add a third-party middleware if we want to use a third-party API gateway service.

Rate limiting can be implemented in the application layer. Middleware is a popular way to implement rate limiting in the application layer. Middleware is a piece of code that intercepts incoming requests. We can add a middleware to our API server to control the rate limiting. Middleware is modular and pluggable. We can add a middleware to our API server without changing any code. We can also add a third-party middleware for rate limiting. We can also add our own middleware if we have the time and resources to do so